

Cumbres triples

La Cordillera Oriental es una cadena montañosa de los Andes que atraviesa Bolivia. Esta consiste de N cumbres, numeradas de 0 a $N - 1$. La **altura** de la cumbre i ($0 \leq i < N$) es $H[i]$, un entero entre 1 y $N - 1$, inclusive.

Para dos cumbres i y j donde $0 \leq i < j < N$, la **distancia** entre ellas se define como $d(i, j) = j - i$.

De acuerdo a antiguas leyendas incas, una terna de cumbres es **mítica** si tiene la siguiente propiedad especial: las alturas de las tres cumbres **coinciden** con sus distancias dos a dos, **ignorando el orden**.

Formalmente, una terna de cumbres (i, j, k) es mítica si

- $0 \leq i < j < k < N$, y
- Las alturas $(H[i], H[j], H[k])$ coinciden con las distancias dos a dos $(d(i, j), d(i, k), d(j, k))$ ignorando el orden.

Por ejemplo, para los índices 0, 1, 2 las distancias dos a dos son (1, 2, 1), así que si las alturas fueran $(H[0], H[1], H[2]) = (1, 1, 2)$, $(H[0], H[1], H[2]) = (1, 2, 1)$, o $(H[0], H[1], H[2]) = (2, 1, 1)$ se consideraría que coinciden, pero si las alturas fueran $(H[0], H[1], H[2]) = (1, 2, 2)$, no coincidirían.

Este problema consiste de dos partes, y cada una tiene sus propias subtareas. Podés resolver las subtareas en cualquier orden. En particular, **no** es necesario completar toda la Parte I antes de intentar la Parte II.

Parte I

Dada la descripción de una cadena montañosa, tu tarea es calcular la cantidad de ternas míticas.

Detalles de Implementación

Debés implementar la siguiente función.

```
long long count_triples(std::vector<int> H)
```

- H : arreglo de longitud N , representando las alturas de las cumbres.
- Esta función se llama exactamente una vez para cada caso de prueba.

La función debe devolver un entero T , el número de ternas míticas en la cadena montañosa.

Restricciones

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$ para cada i tal que $0 \leq i < N$.

Subtareas

La Parte I da un total de 70 puntos.

Subtarea	Puntaje	Restricciones adicionales
1	8	$N \leq 100$
2	6	$H[i] \leq 10$ para cada i tal que $0 \leq i < N$.
3	10	$N \leq 2000$
4	11	Las alturas son no-decrecientes. Esto es, $H[i - 1] \leq H[i]$ para cada i tal que $1 \leq i < N$.
5	16	$N \leq 50\,000$
6	19	Sin restricciones adicionales.

Ejemplo

Considere el siguiente llamado.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Hay 3 ternas míticas en la cadena de montañas:

- Para $(i, j, k) = (1, 3, 4)$, las alturas $(1, 3, 2)$ coinciden con las distancias dos a dos $(2, 3, 1)$.
- Para $(i, j, k) = (2, 3, 6)$, las alturas $(4, 3, 1)$ coinciden con las distancias dos a dos $(1, 4, 3)$.
- Para $(i, j, k) = (3, 4, 6)$, las alturas $(3, 2, 1)$ coinciden con las distancias dos a dos $(1, 3, 2)$.

Por lo tanto, la función debe devolver 3.

Notá que los índices $(0, 2, 4)$ no forman una terna mítica, puesto que las alturas $(4, 4, 2)$ no coinciden con las distancias dos a dos $(2, 4, 2)$.

Parte II

Tu tarea es construir cadenas de montañas con muchas ternas míticas. Esta parte consiste en 6 subtareas de **solo salida** con **puntaje parcial**.

En cada subtarea, te dan dos enteros positivos M y K , y debés construir una cadena montañosa con **a lo sumo** M cumbres. Si tu solución contiene **al menos** K ternas míticas, recibirá todo el puntaje para esta subtarea. En otro caso, tu puntaje será proporcional al número de ternas míticas que contenga tu solución.

Notá que tu solución debe consistir de una cadena montañosa válida. Específicamente, suponé que tu solución tiene N cumbres (N debe satisfacer $3 \leq N \leq M$). Entonces, la altura de la cumbre i ($0 \leq i < N$), denotado $H[i]$, debe ser un entero entre 1 y $N - 1$, inclusive.

Detalles de Implementación

Hay dos métodos para enviar tu solución, y podés usar cualquiera de los dos para cada subtarea:

- **Archivo de salida**
- **Llamada a función**

Para enviar tu solución vía un **archivo de salida**, creá y enviá un archivo de texto en el siguiente formato:

```
N
H[0]  H[1]  ... H[N-1]
```

Para enviar tu solución vía una **llamada a función**, debés implementar la siguiente función.

```
std::vector<int> construct_range(int M, int K)
```

- M : el número máximo de cumbres.
- K : el número deseado de ternas míticas.
- Esta función se llama exactamente una vez para cada subtarea.

La función debe devolver un arreglo H de longitud N , representando las alturas de las cumbres.

Subtareas y puntuación

La Parte II da un total de 30 puntos. Para cada subtarea, los valores de M y K son fijos y dados en la siguiente tabla:

Subtarea	Calificaciòn	M	K
7	5	20	30
8	5	500	2000
9	5	5000	50 000
10	5	30 000	700 000
11	5	100 000	2 000 000
12	5	200 000	12 000 000

Para cada subtarea, si tu solución no es una cadena montañosa válida, tu puntaje será 0 (reportado como `Output isn't correct` en CMS).

En otro caso, sea T el número de ternas míticas en tu solución. Entonces, tu puntaje para la tarea es:

$$5 \cdot \min \left(1, \frac{T}{K} \right).$$

Evaluator local

Las Partes I y II usan el mismo evaluador local, con la distinción entre las dos partes determinada por la primera línea de la entrada.

Formato de entrada para la Parte I:

```
1
N
H[0] H[1] ... H[N-1]
```

Formato de salida para la Parte I:

```
T
```

Formato de entrada para la Parte II:

```
2
M K
```

Formato de salida para la Parte II:

N

H[0] H[1] ... H[N-1]

Notá que la salida del evaluador local cumple con el formato requerido para el archivo de salida en la Parte II.